



Experimenting with **fused compute kernels** in Datafusion

Alexander Rafferty — Rust Sydney 2026



Overview

 Intro to Datafusion and Apache Arrow

 What is a compute kernel?

 How do you fuse them?

 Experiments with Cranelift

 Test results



Datafusion

- An extensible query engine written in Rust
- Uses **Apache Arrow** as its in-memory format
- Extremely performant
- Written in Rust! 🦀



Datafusion



```
1 let results = ctx
2     // Read the source file
3     .read_parquet("data/flights.parquet", Default::default())
4     .await?
5     // Take only the first 10 rows
6     .limit(0, Some(10))?
7     // Execute and collect the results
8     .collect()
9     .await?;
10
11 // Format and print the results
12 let pretty_results = pretty_format_batches(&results)?.to_string();
13 println!("{pretty_results}");
```

Datafusion



```
Compiling fused-compute-kernels v0.1.0 (D:\dev\fused-compute-kernels)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.59s
Running `target\debug\fused-compute-kernels.exe`
```

FL_DATE	DEP_DELAY	ARR_DELAY	AIR_TIME	DISTANCE	DEP_TIME	ARR_TIME
2006-02-19	12	22	159	946	11.683333	14.816667
2006-02-19	105	126	161	946	19.333334	22.6
2006-02-19	77	89	159	946	20.7	23.783333
2006-02-19	7	1	63	370	7.366667	8.766666
2006-02-19	43	38	66	370	10.133333	11.6
2006-02-19	6	5	65	370	10.8	12.266666
2006-02-19	-8	-4	65	370	12.366667	13.9
2006-02-19	18	14	63	370	16.3	17.8
2006-02-19	20	25	64	370	17.833334	19.516666
2006-02-19	0	14	66	370	18.416666	20.283333



Apache Arrow

- Simple, columnar memory format
- Supports arbitrarily nested data
- Enables interoperability between data systems
- Can exploit SIMD operations for excellent performance



Apache Arrow

Arrow arrays

28	0	48	0	79	0
----	---	----	---	----	---

A	I	e	x	B	e	y	o	n	c	é	D	o	n	a	I	d
0	0	0	0	4	0	0	0	11	0	0	0	17	0	0	0	

Name	Age
Alex	28
Beyoncé	44
Donald	79

Logical table



Compute kernels

During query execution, Datafusion needs a way of transforming arrays into new arrays

It does so by combining *pre-compiled* compute kernels that each perform a simple operation, e.g.:

- Arithmetic
- String transforms
- Array indexing
- etc.

Compute kernels

```
(len(name) + age) * 12
```

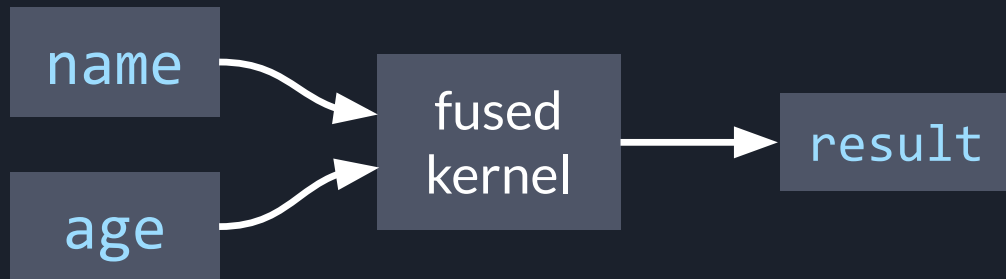




Fused kernels

An alternative approach is to *compile* expressions directly into native machine code

This “fused kernel” can then perform the entire calculation with just one pass over the data





Comparison

Pre-compiled kernels

- ✓ Aggressively optimised
- ✓ Better cache locality
- ✗ Multiple passes over data

Fused kernels

- ✓ One pass over data
- ✓ Consumes less memory
- ✗ Compilation overhead



Experiments with Cranelift

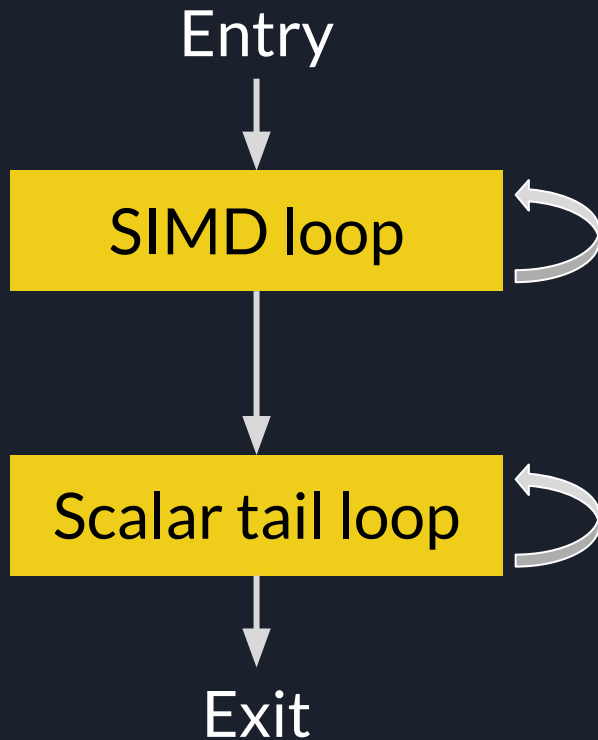
I wanted to explore the potential speed gains that JIT-compiled kernels might offer

I compiled the fused kernels with Cranelift for realistic results

Cranelift is a fast compiler backend written in Rust

I focused on **basic arithmetic** expressions as a starting point

Anatomy of a fused kernel

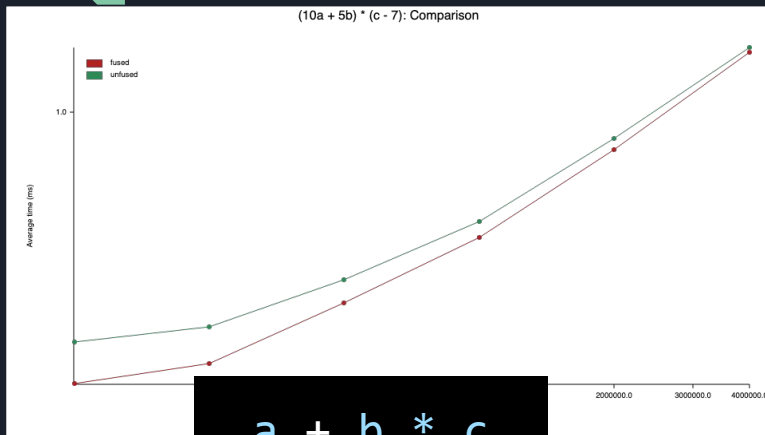


Processes multiple rows at a time
with SIMD instructions like:

`vpadd` `vpsubd` `vpmuld`

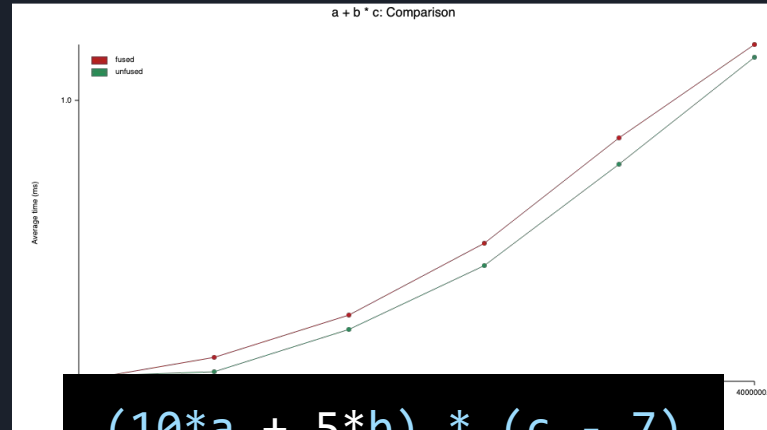
Processes remaining rows
one at a time

Test results



$a + b * c$

Simple expression
Fused kernel was slower



$(10*a + 5*b) * (c - 7)$

Complex expression
Fused kernel was (slightly) faster